



Analyzing Software Evolution through Design Pattern Evolution: A Preliminary Study

Stefania Celenta, Andrea De Lucia, Vincenzo
Deufemia, **Carmine Gravino**, Michele Risi

Software Engineering Lab – SE@SA Lab
Dipartimento di Matematica e Informatica
University of Salerno

Outline

- ❑ Motivations
- ❑ Design pattern recovery process
- ❑ Case study: JHotDraw
- ❑ Conclusions and future work

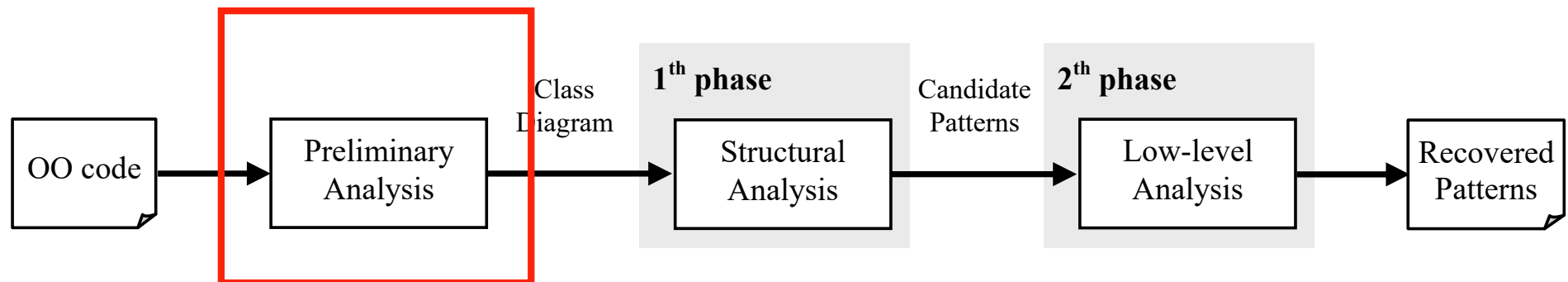
Motivations

- ❑ Design patterns can be used in reverse engineering to capture relevant information on the design and code
 - helping the comprehension of the adopted solution for a system,
 - highlighting wished properties of the design model
 - improving system documentation,
 - guiding the restructuring of the system

- ❑ Can the analysis of the evolution of design pattern instances be useful to understand the decisions made by software engineers in constructing the new versions of a system?

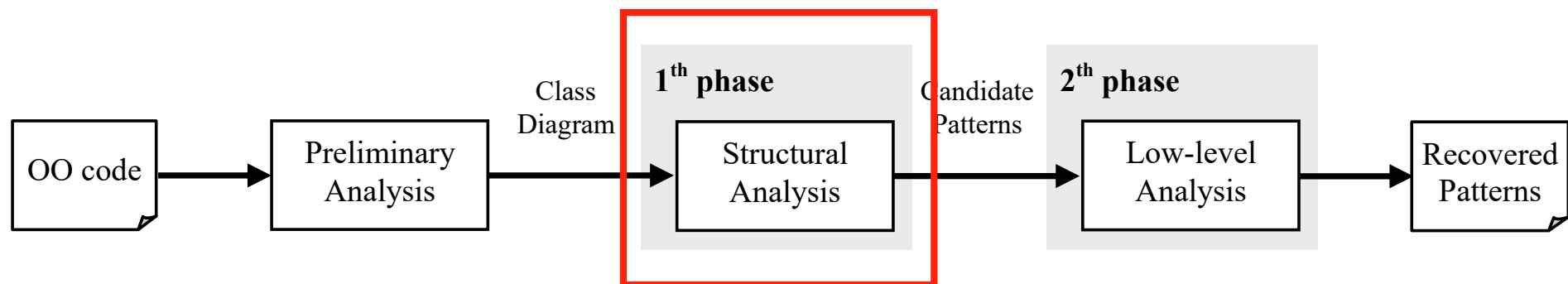
Design pattern recovery process

- Preliminary analysis extracts the structural information proper to recover design patterns and stores them into a database, so that they can be efficiently accessible and navigable.
 - the name of the classes, the name and type of the methods and variables, the name and type of the method parameters, inheritance and association relationships are stored in a suitable data structure



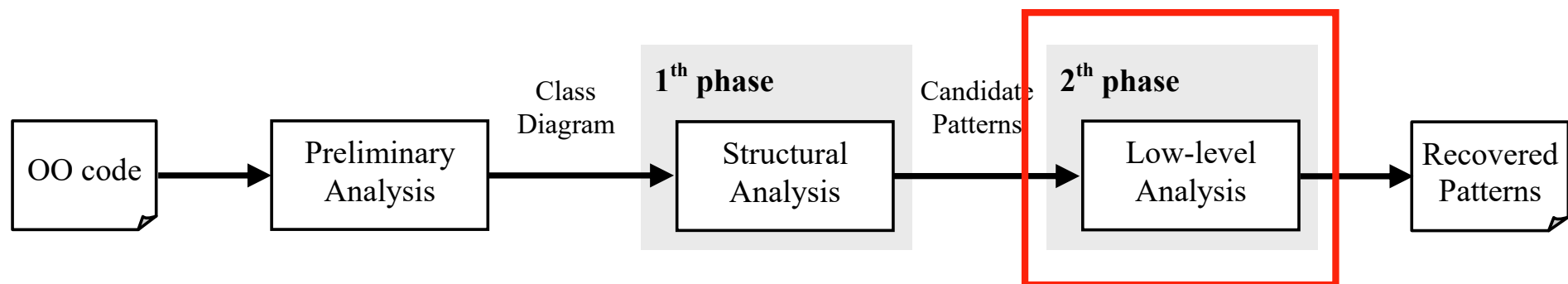
Design pattern recovery process (2)

- The Structural Analysis identifies candidate design patterns by navigating the class diagram stored in the data structure obtained by the previous phase:
 - the identification of patterns is driven by the association and inheritance relationships only
 - and it consists in the application of queries on the data structure representing the input source code



Design pattern recovery process (3)

- The goal of the Low-level Analysis module is to verify that the candidates are real instances of structural design patterns
- It analyzes the stereotypes associated to the classes for validating the role of the participating classes and verifies the method declaration in the delegating classes



Case study: the recovery results

- A preliminary study of evolution of the design patterns, performed exploiting JHotDraw library
 - Identify and analyze kinds of changes in design patterns and relate them to decisions made by developers
- The ratio between the number of classes and the number of pattern instances is decreased from version 5.1 to 5.4b2
 - in versions 5.1, 5.2, and 5.3 the number of classes involved in pattern instances is the 63%, 62%, and 58% of the total number of classes
 - for 5.4b1 and 5.4b2 is 36%.

	5.1	5.2	5.3	5.4b1	5.4b2
#Adapter	41	42	37	54	53
#Bridge	75	81	98	151	166
#Composite	0	0	4	5	5
#Façade	9	9	14	19	20
#Proxy	0	0	0	0	0
#Decorator	0	0	0	0	0

Adapter pattern evolution analysis

- By manually analyzing the recovery results we have considered these cases:
 - pattern instances introduced by adding new classes or by modifying relationships
 - pattern instances eliminated by removing classes or modifying relationships

		from 5.1 to 5.2	from 5.2 to 5.3	from 5.3 to 5.4b1	from 5.4b1 to 5.4b2
Introduced Instances					
by introducing classes		2	12	23	-
# introduced classes		2	13	19	-
role of introduced classes	Adapter	2	8	14	-
	Adaptee	-	7	13	-
	Target	2	5	3	-
by modifying delegations		-	2	1	1
# modified delegations		-	9	2	1
Eliminated Instances					
by eliminating classes		-	-	1	-
# eliminated classes		-	-	1	-
by modifying relationships		1	-	-	2
# modified relationship		1	-	-	2

Evolution analysis

- Relations between the changes in the pattern instances and the decisions made by designers
 - based on the kind of modification involving the patterns

	Insertion of a new pattern instance	Elimination of a pattern instance
Adaptee	feature added	feature removed
Adapter	restructuring	-
Target + Adaptee	feature added	-
Target + Adapter	restructuring	-
Target + Adapter + Adaptee	feature added	-
delegation	restructuring	restructuring

Insertion of an Adapter pattern instance (in Version 5.3)

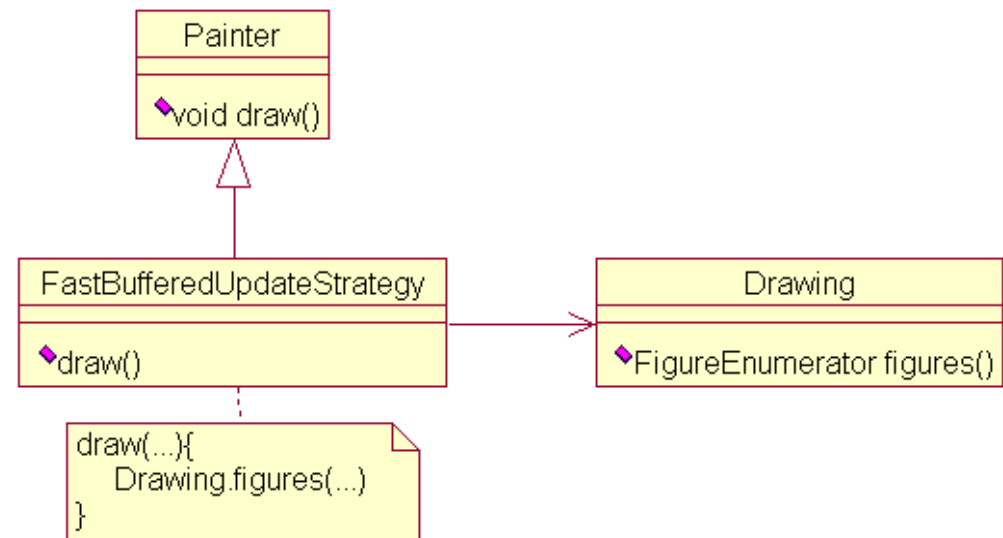
Existing classes:

- Painter: defines the interface for drawing a layer into a DrawingView
- Drawing: is a container for figures (in a layer).

Inserted class:

- FastBufferedUpdateStrategy: introduces a double buffer strategy to draw figures into a DrawingView.

"An existing feature was used to provide a functionality with a customized interface"



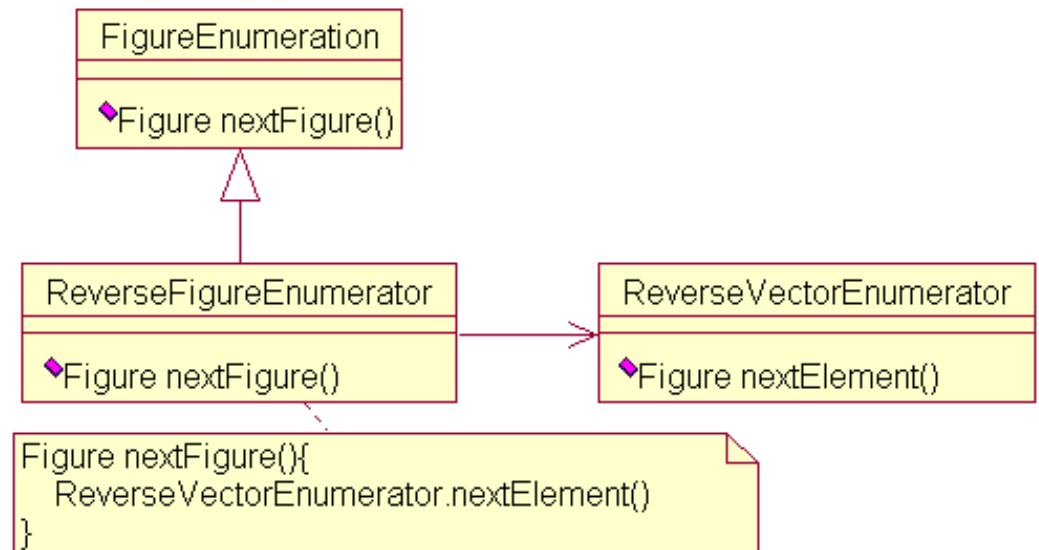
Elimination of an Adapter pattern instance (in version 5.4b1)

- ❑ FigureEnumeration is an interface for Enumerations that access Figures
- ❑ ReverseFigureEnumerator is an Enumeration that enumerates a vector of figures back (size-1) to front (0).

Removed Class:

- ❑ ReverseVectorEnumerator is an Enumeration that enumerates a vector back (size-1) to front (0).

"A feature was removed"



Conclusions

- ❑ Evolution of software systems through the evolution of design patterns
- ❑ Two phase recovery process to detect design pattern instances
- ❑ Case study results: kinds of changes in pattern instances have been used to detect some of the decisions made by developers

Future work

- ❑ a detailed and deeper analysis of the results, also exploiting more documentation and CVS
- ❑ we plan to verify whether there exist relationships between design pattern changes and the reliability of software (as an example, by analyzing bugs)
- ❑ we will also analyze whether or not classes in pattern instances are less change prone
- ❑ more software systems will be investigated to obtain other information on the impact of design patterns on software evolution

Bridge pattern evolution analysis

		from 5.1 to 5.2	from 5.2 to 5.3	from 5.3 to 5.4b1	from 5.4b1 to 5.4b2
Introduced Instances					
by introducing classes		6	27	48	11
# introduced classes		7	20	42	4
Role of Introduced Classes	Abstraction	5	19	41	11
	Implementor	-	5	7	-
	Concrete Implementor	3	11	13	-
by modifying delegations		1	4	11	8
# modified delegations		1	22	24	13
Eliminated Instances					
by eliminating classes		-	-	1	-
# eliminated classes		-	-	1	-
by modifying relationships		1	14	5	4
# modified relationship		1	46	17	17